



CLAUDE CODE · AVANÇADO · AULA 7

/loop

Tarefas agendadas

Agendamento nativo do Claude Code — execute qualquer tarefa em intervalos definidos, sem cron externo, sem scripts, sem CI/CD. Bloco 2: Fluxos Avançados.

--interval

--cron

/schedule

gerenciamento



Tarefas que precisam *repetir*

😓 Você faz manualmente

- ✗ Checar testes após cada commit
- ✗ Verificar deps desatualizadas
- ✗ Revisar logs de erro
- ✗ Backup de arquivos críticos

⚠️ Você esquece

- ✓ Rodar o linter antes do PR
- ✓ Checar `npm audit`
- ✓ Limpar temporários
- ✓ Sincronizar docs

✅ Com `/loop`, sozinho

- ✓ A cada 30min: testes + falhas
- ✓ A cada hora: deps + audit
- ✓ A cada commit: lint + coverage
- ✓ Toda segunda: relatório semanal



`/loop` transforma qualquer tarefa manual e repetitiva em automação periódica **nativa** do Claude Code — sem cron externo, sem scripts shell, sem CI/CD.

O que é */loop*?

O sistema de agendamento nativo do Claude Code. Executa qualquer tarefa **em intervalos definidos** — de segundos a expressões cron — sem ferramentas externas.

```
/loop --interval 30m "Rode npm test. Se falhar, mostre quais quebraram e o motivo."
```

`/loop` Comando principal de agendamento

`--interval 30m` Intervalo de execução (a cada ciclo)

`"tarefa..."` A instrução executada a cada disparo

`--interval 30s` → 30 segundos

`--interval 5m` → 5 minutos

`--interval 2h` → 2 horas

`--interval 1d` → 1 dia

`--cron "0 9 * * 1"` → seg 9h

`--on-change src/` → ao salvar

Como um loop *roda*

1 ▶ Início
Registra o job e mostra a próxima execução

2 ⌚ Aguarda intervalo
Monitora o clock, sem consumir CPU

3 ⚡ Executa
Roda a instrução com contexto completo

4 📊 Processa
Analisa: sucesso, falha ou aviso

5 📧 Reporta
Saída formatada, notifica via hooks

6 🔄 Reagenda
Calcula o próximo horário e volta ao passo 2

↺ Loop até `/loop stop` ou Ctrl+C

Gerenciando seus *loops*

Criar

`/loop --interval 30m "tarefa"` — cada loop recebe um ID (loop-1, loop-2...).

Pausar

`/loop pause loop-1` — pausa sem destruir; o ID é preservado.

Parar

`/loop stop loop-1` — destrói e libera recursos; ID não reutilizável.

Listar

`/loop list` — ID, intervalo, próxima execução, status e última saída.

Retomar

`/loop resume loop-1` — reagenda a partir do resume.

Rodar agora

`/loop run loop-1` — executa já, sem esperar o intervalo. Ideal para testar.

/schedule *no futuro*



/loop — recorrente

Executa **infinitamente** a cada intervalo. Para monitoramento e manutenção.

```
/loop --interval 1h "checar logs de erro"
```



/schedule — única vez

Executa **uma vez** num horário futuro. Para deploys e relatórios com janela.

```
/schedule "2025-12-31 23:50" "deploy release"
```

"today 23:59"

Relatório de commits do dia ao fim do expediente

"next friday 17:00"

npm audit + teste de carga + relatório semanal

"tomorrow 09:00"

PRs abertos há > 3 dias — morning standup prep

"+2h"

Verificar staging + smoke tests (delay pós-deploy)

Cron jobs com *--cron*

Aceita expressões cron padrão de 5 campos para agendamentos sofisticados.

0 9 * * 1

minuto

0-59

hora

0-23

dia mês

1-31

mês

1-12

dia sem.

0=Dom

0 9 * * 1 seg 9h

0 */2 * * * a cada 2h

30 8 * * 1-5 úteis 8:30

0 3 * * * madrugada 3h

0 17 * * 5 sexta 17h

*/15 * * * * a cada 15min

0 12 1 * * dia 1 do mês

0 0 * * 0 domingo 0h

/loop no *mundo real*



Vigia de segurança

--interval 1h · lista pacotes vulneráveis + links de CVEs



Perf. de testes

--cron "* / 30 9-18 * * 1-5" · % cobertura e
tendência



Sync de docs

--on-change src/ · docs atualizadas a cada mudança



Health check

--interval 15m · notifica via hook no Slack se falhar



Relatório diário

--cron "0 18 * * 1-5" · resumo ao fim do expediente



Limpeza noturna

--cron "0 3 * * *" · notifica se liberar > 1GB

/loop + *Hooks*

O /loop **detecta** o problema; o Hook Notification (Aula 2) **comunica** onde você preferir.

```
# /loop com instrução inteligente
/loop --interval 30m \
  "Rode npm test e npm audit." \
  "Se falhar ou houver CVE high/critical:" \
  "1. Identifique o problema" \
  "2. Sugira a correção" \
  "3. Use notify: CRÍTICO △ [detalhes]"
```

```
# notify-slack.sh (Hook Notification)
INPUT=$(cat)
MSG=$(echo "$INPUT" | jq -r '.message')

# só notifica se for crítico
if echo "$MSG" | grep -q "CRÍTICO"; then
  curl -s -X POST "$SLACK_HOOK" \
    -d "{\"text\":\"△ $MSG\"}"
fi
```

💡 Resultado: alerta no Slack só quando algo está errado — sem spam a cada execução

Boas práticas *com /loop*

✓ Faça

- ✓ Seja **específico** no formato de saída
- ✓ Use `/loop run` antes de agendar
- ✓ Configure um **intervalo sensato** (evite $< 5\text{min}$)
- ✓ Combine com **Hooks** para alertas
- ✓ Documente os loops ativos num `LOOPS.md`

✗ Evite

- ✗ Intervalos **curtos demais** sem necessidade
- ✗ Instruções **vagas** ("cheque se está tudo bem")
- ✗ Loops que **escrevem em arquivos** sem cuidado
- ✗ Esquecer de **parar loops** ao mudar de projeto
- ✗ Depender de loops **sem alertas** configurados

Teste *rápido*

Você quer rodar `npm audit` toda segunda-feira às 9h, repetidamente. Qual comando?

A `/schedule "next monday 09:00" "npm audit"`

B `/loop --cron "0 9 * * 1" "npm audit"`

C `/loop --interval 1d "npm audit"`

D `/loop --cron "9 0 1 * *" "npm audit"`

Resposta: B. Precisa ser **recorrente** (logo `/loop`, não `/schedule` que roda uma vez só) e num **dia/horário fixo** (logo `--cron`). A expressão `0 9 * * 1` = minuto 0, hora 9, qualquer dia/mês, dia-da-semana 1 (segunda). A opção D inverte os campos.



RESUMO DA AULA 7

Automação que *roda sozinha*



/loop

Execução periódica nativa: `--interval` (30s...1d) e `--cron`. Zero dependência externa.



/schedule

Execução única no futuro: datas absolutas, relativas (+2h) e linguagem natural.



/loop + Hooks

Alertas inteligentes: só notifica quando detecta problema — sem spam.



Gerenciamento

`list` • `pause` • `resume` • `stop` • `run` — controle total sem reiniciar.

Fim do módulo **2 • Avançado** — Hooks, Sub-Agents, Agent Teams e /loop.